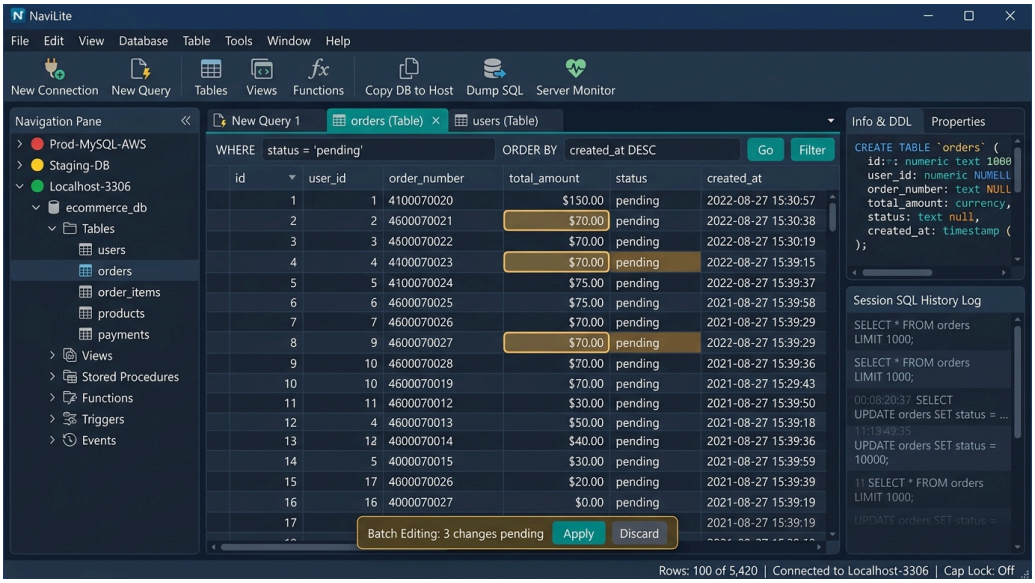
Spesifikasi Fungsional & Teknis Detail Fitur (FSD): NaviLite

Rincian Komponen UI (C# .NET 8), Kueri Internal MySQL, Kontrak FFI (Rust Core), serta Penanganan Edge Case untuk 8 Modul Utama

Dokumen ini menjabarkan spesifikasi fungsional dan teknis secara menyeluruh untuk setiap fitur yang dibangun pada **NaviLite**—klien manajemen database MySQL/MariaDB desktop *native* berbasis **C# (.NET 8)** dan **Rust Core Engine** (`mysql_async` + `csbindgen`) yang menggabungkan tata letak ruang kerja Navicat Premium dengan mesin transfer data lintas server milik SQLyog.



Gambar 1: Referensi Antarmuka Utama NaviLite — Menyatukan Main Toolbar Ringkas, Navigation Pane Berwarna, Virtualized Data Grid dengan Batch Editing, serta Panel Info & DDL.

Modul 1: Connection & Session Manager (`ConnectionManager`)

Modul ini menangani siklus hidup profil koneksi database, enkripsi kredensial di tingkat sistem operasi, pembukaan terowongan jaringan (*SSH/SSL*), serta manajemen *connection pool* terpisah antara operasi antarmuka dan eksekusi kueri pengguna.

1.1 Rincian Sub-Fitur, Parameter Input & Mekanisme Sistem

Sub-Fitur	Komponen UI & Input Pengguna (C#)	Implementasi Mesin Rust & Protokol MySQL
Koneksi Standar TCP/IP & Unix Socket	Tab *General*: `Connection Name`, `Host/IP` (default `127.0.0.1`), `Port` (`3306`), `Username`, `Password`, `Save Password` checkbox, serta opsi `Unix Socket Path`.	Membangun struktur `mysql_async::OptsBuilder` dengan `tcp_nodelay(true)`, `conn_ttl(300s)`, dan `init(vec!["SET NAMES utf8mb4 COLLATE utf8mb4_unicode_ci"])`.

Sub-Fitur	Komponen UI & Input Pengguna (C#)	Implementasi Mesin Rust & Protokol MySQL
In-Memory SSH Tunneling	Tab *SSH*: Checkbox `Use SSH Tunnel`, `SSH Host`, `SSH Port` (22), `SSH User`, dan metode autentikasi (`Password` atau `Private Key` `.pem/.ppk` + `Passphrase`).	Menggunakan crate `russh` untuk membuat saluran *direct-tcpip port forwarding* langsung di memori Rust menuju `MySQL Host:Port` tanpa membuka *port* lokal yang terekspos ke proses lain.
Enkripsi SSL/TLS & Sertifikat	Tab *SSL*: Checkbox `Use SSL`, `Verify Server Certificate`, serta pemilih berkas `CA Certificate`, `Client Certificate`, dan `Client Key`.	Mengonfigurasi `mysql_async::SslOpts` menggunakan `rustls` (mendukung sertifikat RSA maupun ECC sesuai standar Navicat 17.3+).
Environment Color Badge & Guardrail	Pemilih warna lingkungan koneksi: **Merah (Production)**, **Kuning (Staging)**, dan **Hijau (Local/Dev)**.	Jika koneksi berlabel **Merah (Production)**, *parser* SQL di Rust mendeteksi perintah `DROP`, `TRUNCATE`, serta `DELETE`/`UPDATE` tanpa klausa `WHERE` dan memaksa dialog konfirmasi ganda di C#.
OS Keychain & Ekspor Koneksi	Menu *File → Export/Import Connections* (`.nvconn.json` atau kompatibel `.ncx` Navicat).	Metadata koneksi disimpan di `connections.json`, sedangkan kata sandi MySQL/SSH disimpan di *Windows Credential Manager (DPAPI) / macOS Keychain / Linux Secret Service* via crate `keyring`.

1.2 Arsitektur Dual-Pool & Kontrak FFI Koneksi

- **Dual Session Pool per Koneksi Aktif**: Setiap koneksi yang dibuka di *Navigation Pane* menginisialisasi dua *pool* di Rust: (1) **Metadata & UI Pool** (maksimal 2 koneksi ringan untuk *Navigation Tree*, *DDL Preview*, dan *Autocomplete*) agar pembacaan skema tidak pernah terblokir oleh kueri lambat, serta (2) **Interactive Workload Pool** (koneksi khusus per *tab Data Editor* atau *Query Editor* agar variabel sesi `@var` dan transaksi `START TRANSACTION` tidak tercampur antar-tab).
- **Kontrak FFI Koneksi**: `navilite_conn_test(cfg_json: *const c_char) -> FfiResult` (menguji jabat tangan jaringan & mengembalikan versi server MySQL/MariaDB serta latensi dalam milidetik), `navilite_conn_open(conn_id: u32) -> FfiResult`, dan `navilite_conn_close(conn_id: u32) -> i32`.

Modul 2: Lazy-Load Schema & Object Navigation Tree (`NavigationPane`)

Mengelola pohon hierarki objek di sisi kiri aplikasi menggunakan teknik pemuatan bertahap (*lazy-loading*) seperti pada Navigation Pane Navicat agar koneksi dengan ratusan database dan ribuan tabel tetap terbuka dalam hitungan milidetik.

2.1 Kueri Metadata Internal & Pohon Hierarki

Level Simpul Pohon (Node)	Kueri MySQL Internal Saat Simpul Diperluas (Expand)	Informasi Visual & Aksi Klik Kanan (Context Menu)
Level 1: Daftar Database	`SHOW DATABASES;` dikombinasikan dengan `SELECT SCHEMA_NAME, DEFAULT_CHARACTER_SET_NAME, DEFAULT_COLLATION_NAME FROM information_schema.SCHEMATA;`	Menampilkan ikon silinder database; klik kanan menyediakan: *Open Database*, *New Query*, *Create/Alter/Drop Database*, **Copy Database to Different Host/DB...**, dan *Dump Database as SQL...*.
Level 2: Kategori Objek (*Tables, Views, Procedures, Functions, Triggers, Events*)	Tabel & View: `SELECT TABLE_NAME, TABLE_TYPE, ENGINE, TABLE_ROWS, DATA_LENGTH, TABLE_COMMENT FROM information_schema.TABLES WHERE TABLE_SCHEMA = ?;` Rutin: `SELECT ROUTINE_NAME, ROUTINE_TYPE, DATA_TYPE FROM information_schema.ROUTINES WHERE ROUTINE_SCHEMA = ?;`	Menampilkan jumlah objek dalam tanda kurung (misal `Tables (42)`); klik ganda pada tabel membuka *Virtualized Data Grid*, sedangkan klik kanan menyediakan *Design Table* (`Ctrl+D`), **Copy Table(s) to Different Host/DB...**, *Import/Export*, *Truncate*, dan *Drop*.
Level 3: Sub-Elemen Tabel (*Columns, Indexes, Foreign Keys*)	`SHOW FULL COLUMNS FROM` `` `db`.`tbl` ``; `SHOW INDEX FROM` `` `db`.`tbl` ``; `SELECT CONSTRAINT_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME FROM information_schema.KEY_COLUMN_USAGE WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?;`	Memudahkan inspeksi nama kolom, tipe data (misal `BIGINT UNSIGNED`), dan ikon kunci (*Primary Key* emas, *Foreign Key* biru) langsung dari pohon navigasi tanpa harus membuka jendela *Table Designer*.

2.2 Indeks Schema Cache & Pencarian Instan

- **In-Memory Radix/Trie Index di Rust**: Begitu sebuah database dibuka, Rust memuat daftar nama tabel, *view*, kolom, dan prosedur di latar belakang ke dalam struktur `SchemaCache`. Struktur ini melayani pencarian instan di kotak filter *Navigation Pane* (menyembunyikan simpul yang tidak cocok saat pengguna mengetik) sekaligus menjadi sumber data *Autocomplete* pada *SQL Query Editor*.

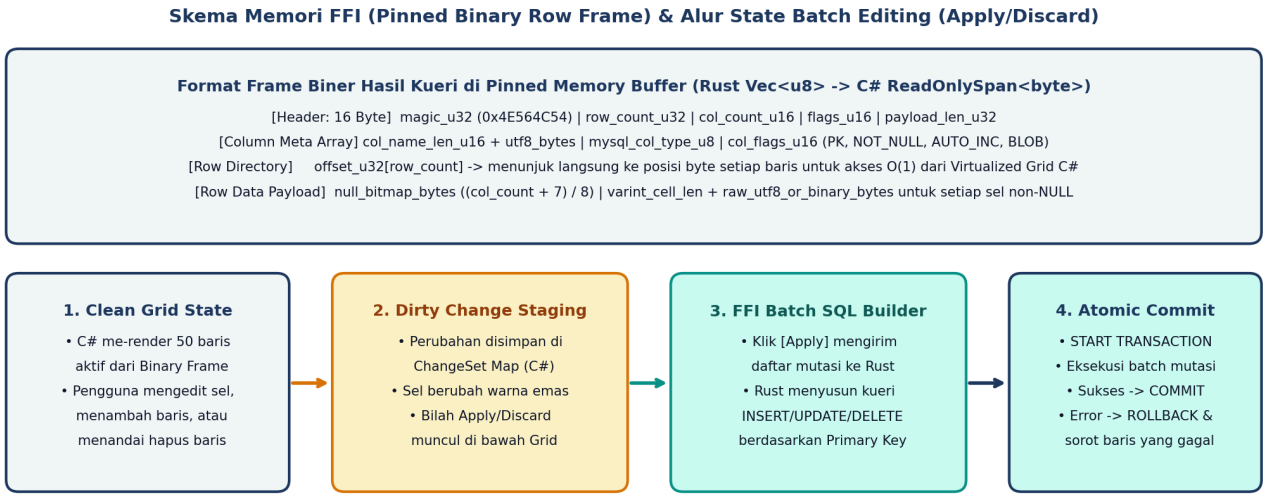
Modul 3: Virtualized Data Grid, Form View & Batch Editor (`DataEditor`)

Menyediakan antarmuka penjelajahan dan penyuntingan rekaman tabel berkecepatan tinggi yang mampu menampilkan ribuan baris tanpa jeda *Garbage Collection*, dilengkapi mekanisme **Batch Editing** seperti pada Navicat 18.

3.1 Protokol Pinned Binary Row Buffer & Virtualisasi UI

- **Zero-Copy Binary Frame via FFI**: Alih-alih mengubah hasil kueri menjadi array objek atau JSON, Rust menyusun halaman data (default `LIMIT 0, 1000`) ke dalam blok memori kontigu `Vec<u8>` yang memiliki direktori *byte offset* per baris (`offset_u32[row_count]`). C# membaca blok ini melalui `ReadOnlySpan<byte>` hanya untuk ~50 baris yang sedang terlihat di layar (*Viewport Virtualization*), menghasilkan pengguliran 60 FPS yang mulus.

- **Bilah Ekspresi WHERE & ORDER BY Cepat**: Di bagian atas *Data Grid* terdapat dua kotak input teks dengan *autocomplete* nama kolom: kotak `WHERE` (misal `status = 'pending' AND total_amount > 50`) dan kotak `ORDER BY` (misal `created_at DESC`). Mengeklik header kolom juga mengganti urutan `ASC` → `DESC` → `Default` secara otomatis.



Gambar 2: Struktur Memori Pinned Binary Row Frame untuk Transfer Data Zero-Copy (Rust → C#) dan Mesin Status Batch Editing (Apply/Discard).

3.2 Mekanisme Batch Editing (`Apply` / `Discard`) & Penyunting Sel Khusus

Komponen / Fitur Editor	Perilaku Antarmuka Pengguna (C#)	Pembangkitan SQL & Penanganan Transaksi (Rust)
Batch Dirty-Cell Staging	Saat pengguna mengubah nilai sel, menambah baris baru (+), atau menandai baris untuk dihapus (-), perubahan ditandai dengan bingkai warna emas dan bilah mengambang **Batch Editing: N changes pending [Apply] [Discard]** muncul di bawah *grid*.	Saat tombol **Apply** ditekan, Rust membuka `START TRANSACTION;`, mengeksekusi seluruh pernyataan `INSERT`, `UPDATE ... WHERE pk = ?`, dan `DELETE ... WHERE pk = ?` secara berurutan, lalu menjalankan `COMMIT;`. Jika satu baris gagal (misal melanggar *constraint*), Rust menjalankan `ROLLBACK;` dan mengembalikan nomor baris yang gagal ke C#.
Specialized Cell Editors	Klik ikon *Cell Editor* atau tekan `F2` pada sel untuk membuka pop-up kontekstual: (1) **Text/Code Editor**, (2) **JSON Tree & Formatter**, (3) **Hex/BLOB Viewer** (lengkap dengan tombol *Load File* / *Save to Disk*), dan (4) **Date/Time Picker**.	Mendukung pintasan klik kanan `Set to NULL` (`Ctrl+Alt+N`), `Set to Empty String ('')`, dan `Set to DEFAULT`. Untuk kolom `BLOB` besar (> 64 KB), Rust hanya mengambil pratinjau 256 byte pertama di *grid* dan memuat *stream* penuh saat *Hex/Image Viewer* dibuka.
Foreign Key Data Picker	Pada kolom yang memiliki relasi *Foreign Key*, terdapat tombol *dropdown lookup* yang menampilkan daftar rekaman dari tabel induk beserta kolom deskriptifnya.	Rust mengeksekusi `SELECT` ``pk``, ``display_col`` `FROM` ``parent_tbl`` `WHERE` ``pk`` `LIKE ? LIMIT 100;` secara asinkron saat pengguna mencari nilai referensi.

Komponen / Fitur Editor	Perilaku Antarmuka Pengguna (C#)	Pembangkitan SQL & Penanganan Transaksi (Rust)
Form View & Side Record Navigator	Tombol `Switch to Form View` (`Ctrl+G`) menampilkan satu baris sebagai formulir vertikal dua kolom, didampingi panel daftar rekaman di samping untuk melompat antar-baris tanpa mengeklik *Next* berulang kali.	Membaca langsung dari *offset* baris yang sama pada `Pinned Binary Row Frame` tanpa perlu menjalankan kueri ulang ke server MySQL.
Copy Rows As & Session History Log	Menyalin baris terpilih di *grid* menjadi pernyataan `INSERT INTO`, `UPDATE`, format `CSV`, `TSV` (siap tempel ke Excel), atau array `JSON`.	Setiap kueri pembacaan data, pemfilteran, dan transaksi perubahan dicatat beserta waktu eksekusinya ke panel **Session SQL History Log** di kanan bawah.

Modul 4: Visual Table & Object Designer (`ObjectDesigner`)

Memungkinkan pembuatan tabel baru maupun modifikasi skema tabel yang sudah ada melalui antarmuka grafis tanpa menulis DDL secara manual, didukung oleh mesin pembanding skema (*Schema Diff*) di Rust.

4.1 Spesifikasi 6 Sub-Tab pada Table Designer

- **Tab 1 — Columns**: Grid penyuntingan kolom berisi `Name`, `Type` (`INT`, `BIGINT`, `VARCHAR`, `TEXT`, `DECIMAL`, `DATETIME`, `TIMESTAMP`, `JSON`, `ENUM`, `BLOB`, dll.), `Length/Values`, `Decimals`, checkbox `Not Null`, `Unsigned`, `Auto Increment`, `Zero Fill`, input `Default Value` (mendukung `CURRENT_TIMESTAMP` dan `ON UPDATE CURRENT_TIMESTAMP`), opsi `Virtual / Stored Generated Expression`, `Charset/Collation` tingkat kolom, dan `Comment`. Tersedia tombol *Move Up / Move Down* yang menghasilkan klausa `FIRST` atau `AFTER `col_name``.
- **Tab 2 — Indexes**: Mengelola indeks `PRIMARY`, `UNIQUE`, `NORMAL (BTREE)`, `FULLTEXT`, dan `SPATIAL`, termasuk pemilihan multi-kolom (*composite index*), panjang awalan (*sub-part key length*), urutan `ASC/DESC` (MySQL 8.0+), dan komentar indeks.
- **Tab 3 — Foreign Keys**: Mengatur `Constraint Name`, kolom anak (*referencing columns*), database & tabel induk (*referenced table*), kolom induk, serta aksi `ON DELETE` dan `ON UPDATE` (`CASCADE`, `SET NULL`, `RESTRICT`, `NO ACTION`).
- **Tab 4 — Triggers**: Merancang *trigger* yang terikat pada tabel aktif dengan pilihan waktu (`BEFORE` / `AFTER`), kejadian (`INSERT` / `UPDATE` / `DELETE`), urutan eksekusi (`FOLLOWS` / `PRECEDES`), dan editor kode tubuh `BEGIN ... END`.
- **Tab 5 — Options**: Mengatur properti tingkat tabel meliputi `Storage Engine` (`InnoDB`, `MyISAM`, `MEMORY`, `ARCHIVE`, `CSV`), `Default Character Set`, `Default Collation`, nilai awal `Auto Increment`, `Row Format` (`DYNAMIC`, `COMPRESSED`, `COMPACT`), dan komentar tabel.
- **Tab 6 — SQL Preview (Live DDL Diff)**: Menampilkan pernyataan `CREATE TABLE` (untuk tabel baru) atau skrip diferensial `ALTER TABLE `tbl` CHANGE COLUMN ..., ADD INDEX ..., DROP FOREIGN KEY ...;` secara langsung sebelum pengguna menekan tombol **Save** (`Ctrl+S`).

4.2 Perancang View, Stored Procedure, Function & Event

- **View Designer**: Menyediakan editor definisi kueri `SELECT` beserta pengaturan `Algorithm` (`UNDEFINED`, `MERGE`, `TEMPTABLE`), `SQL Security` (`DEFINER` / `INVOKER`), dan `Check Option` (`CASCADED` / `LOCAL`).
- **Routine & Event Designer**: Menyediakan *template* parameter (`IN` / `OUT` / `INOUT`), tipe pengembalian (`RETURNS` untuk *Function*), sifat deterministik (`DETERMINISTIC` / `READS SQL DATA`), serta pengatur jadwal `ON SCHEDULE AT / EVERY` untuk MySQL Event Scheduler.

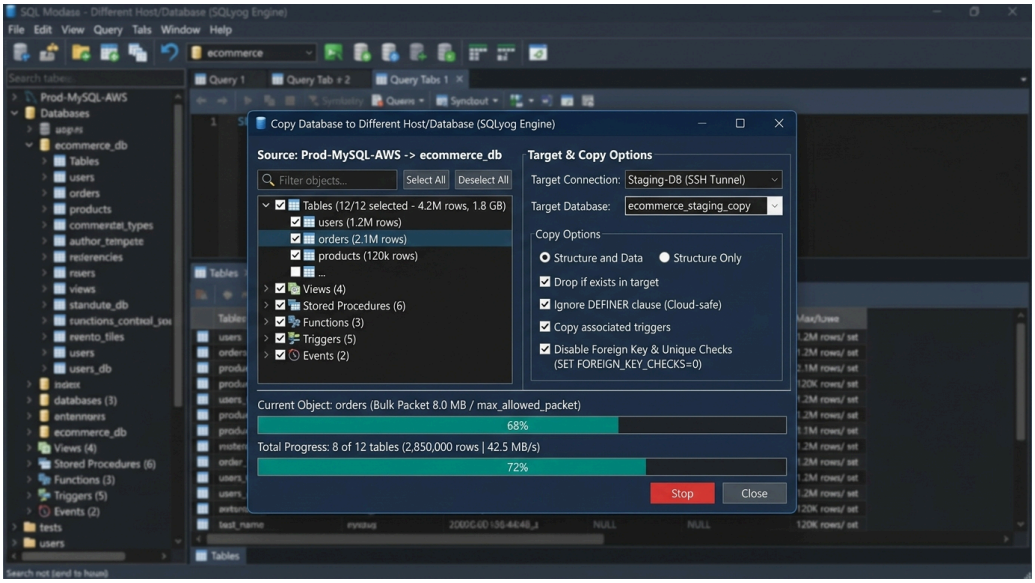
Modul 5: SQL Query Editor & Schema-Aware Autocomplete (`QueryEditor`)

Menyediakan lingkungan penulisan dan pengoptimalan kueri SQL yang cepat, dilengkapi mesin *lexer* dan *autocomplete* di sisi Rust yang memahami alias tabel serta struktur database aktif.

Fitur Query Editor	Mekanisme Antarmuka & Pintasan (C#)	Pemrosesan Parser & Eksekusi di Sisi Rust
Schema-Aware Code Completion	Muncul otomatis saat mengetik atau menekan `Ctrl+Space`; menampilkan ikon pembeda untuk Keyword, Tabel, Kolom, View, dan Fungsi.	Menganalisis kueri di sekitar kursor untuk memetakan alias tabel (misal pada `SELECT o.| FROM orders o`, Rust langsung menyarankan daftar kolom milik tabel `orders` dari `SchemaCache`).
Eksekusi Kueri Fleksibel & Statement Splitter	Tombol **Run All** (`Ctrl+F9`) menjalankan seluruh isi editor; tombol **Run Current** (`F9`) hanya mengeksekusi satu pernyataan SQL tempat kursor berada atau blok teks yang diblok.	Pemisah pernyataan (*Statement Splitter*) di Rust memahami komentar (`--`, `/* */`), literal string ber-*escape*, serta direktif `DELIMITER $$` sehingga pembuatan *Stored Procedure* dan *Trigger* berjalan mulus.
Multi-Result Grid & Pin Result Tab	Setiap pernyataan `SELECT` menghasilkan tab hasil di panel bawah; pengguna dapat mengeklik ikon semat (**Pin Result**) pada tab hasil tertentu.	Tab hasil yang disematkan (*pinned*) dikunci di memori sehingga tidak tertimpa saat pengguna memodifikasi dan menjalankan ulang kueri untuk membandingkan hasil sebelum dan sesudah perubahan.
SQL Beautifier & EXPLAIN Analyzer	Tombol **Format SQL** (`Ctrl+Shift+F`) merapikan indentasi kueri; tombol **Explain** (`Ctrl+E`) menampilkan analisis rencana eksekusi.	Menjalankan `EXPLAIN` dan `EXPLAIN FORMAT=JSON` pada MySQL, lalu menandai operasi berbiaya tinggi (seperti `ALL / Full Table Scan`, `Using filesort`, atau `Using temporary`) dengan indikator warna peringatan.

Modul 6: Cross-Connection Data Transfer — Mekanisme SQLyog (`DataTransferEngine`)

Modul unggulan ini mengimplementasikan fitur **Copy Database/Table to Different Host/Database** menggunakan desain satu layar (*Single-Dialog UX*) dan arsitektur *streaming bulk packetizer* yang diadaptasi dari SQLyog.



Gambar 3: Antarmuka Single-Dialog Copy Database to Different Host/Database pada NaviLite dengan Pohon Objek Sumber, Opsi Sanitasi DEFINER/FK, serta Progres Streaming Bulk Packet.

6.1 Spesifikasi Komponen UI & Logika Validasi Opsi

- **Pemilihan Objek Sumber Terstruktur**: Menampilkan pohon objek dari koneksi sumber yang dikelompokkan dalam 6 simpul induk: `Tables` (menampilkan estimasi baris dan ukuran dari `information_schema.TABLES`), `Views`, `Stored Procedures`, `Functions`, `Triggers`, dan `Events`. Jika pengguna membuka dialog dari klik kanan pada satu tabel (*Copy Table to Different Host/DB*), hanya tabel tersebut yang otomatis dicentang saat dialog terbuka.
- **Konfigurasi Target & Pembuatan Database Otomatis**: Pengguna memilih koneksi tujuan dari *dropdown* (dapat berupa server lokal, server jarak jauh via SSH Tunnel berbeda, atau server yang sama dengan nama database berbeda). Pada *editable combobox* `Target Database`, jika pengguna mengetikkan nama database yang belum ada di server tujuan, mesin Rust otomatis mengeksekusi `CREATE DATABASE IF NOT EXISTS `target_db` CHARACTER SET ... COLLATE ...;` sesuai konfigurasi database sumber.
- **Matriks Perilaku 5 Opsi Penyalinan**:
  1. **Structure and Data vs. Structure Only**: Menentukan apakah Tahap 3 (*Streaming Bulk Packetizer*) dijalankan atau dilewati setelah pembuatan DDL tabel.
  2. **Drop if exists in target**: Menambahkan `DROP TABLE/VIEW/PROCEDURE/FUNCTION/TRIGGER/EVENT IF EXISTS` di server target sebelum mengeksekusi pernyataan `CREATE`.
  3. **Ignore DEFINER clause (Cloud-safe)**: Menjalankan pembersih ekspresi reguler di Rust pada seluruh DDL untuk menghapus `DEFINER=`[^`]+`@`[^`]+` dan mengubah `SQL SECURITY DEFINER` menjadi perilaku *default* pengguna target agar tidak memicu error `ERROR 1227 (42000): Access denied; you need SUPER or SET_USER_ID privilege` di layanan *cloud* (AWS RDS, Azure MySQL, Google Cloud SQL).
  4. **Copy associated triggers**: Menyinkronkan status centang *trigger* secara otomatis saat tabel induknya dicentang atau dihapus centangnya di pohon objek (SQLyog KB #115).
  5. **Disable Foreign Key & Unique Checks**: Mengeksekusi `SET FOREIGN_KEY_CHECKS = 0; SET UNIQUE_CHECKS = 0;` pada sesi target sebelum transfer dimulai dan memulihkannya (`= 1`) di blok `finally` setelah selesai (SQLyog Issue #2315).

6.2 Detail Algoritma Pipeline Streaming 4-Tahap di Rust

Untuk menjamin kecepatan transfer maksimal tanpa lonjakan pemakaian RAM pada tabel berukuran besar, mesin Rust membagi eksekusi transfer ke dalam tahapan pipeline asinkron berikut:

Komponen Mesin Rust	Algoritma & Struktur Data Internal	Penanganan Kasus Khusus (Edge Cases)
1. Dynamic Packet Size Negotiator	Menjalankan `SELECT @@max_allowed_packet, @@net_buffer_length;` pada server target. Menetapkan ukuran *target chunk*: `chunk_limit = clamp(target_max_packet * 3 / 4, 1_048_576, 16_777_216)` (antara 1 MB hingga 16 MB).	Jika satu baris tunggal (misal berisi dokumen `LONGBLOB` 20 MB) melebihi `target_max_allowed_packet`, mesin mencoba menaikkan `SET GLOBAL max_allowed_packet` jika berhak, atau mencatat peringatan spesifik nama tabel dan kunci utama baris tersebut.
2. DDL Extractor & Column Filter	Mengeksekusi `SHOW CREATE TABLE` `` `src`.`tbl` `` dan mem-*parsing* definisi kolom menggunakan `SHOW COLUMNS FROM` `` `src`.`tbl` `` untuk memisahkan kolom biasa dari kolom `VIRTUAL GENERATED` / `STORED GENERATED`.	Kolom *generated* dikecualikan dari daftar kolom `INSERT INTO` `` `tbl` (`c1`, `c2`) `` `VALUES ...` karena MySQL menolak penyisipan nilai eksplisit ke kolom *generated* dan akan menghitung ulang nilainya secara otomatis di server target.
3a. Unbuffered Stream Reader (Producer)	Membaca `SELECT` `` `c1`, `c2`... `` `FROM` `` `src`.`tbl` `` menggunakan *unbuffered streaming cursor* (`mysql_async::Conn::query_iter`) dan mengirimkan baris ke kanal `tokio::sync::mpsc::channel(4)`.	Kapasitas kanal 4 paket memberlakukan *backpressure* otomatis: jika jaringan menuju server target lebih lambat daripada pembacaan sumber, *Producer* otomatis menunggu sehingga pemakaian RAM terkunci di bawah 85 MB.
3b. Extended Bulk Packetizer (Consumer)	Merakit ribuan baris dari kanal ke dalam satu `Vec<u8>` berformat `INSERT INTO` `` `tbl` `` `VALUES (...),(...);` hingga mendekati `chunk_limit`, lalu mengeksekusinya ke server target.	Nilai `NULL` ditulis sebagai literal `NULL`, string di-*escape* untuk karakter `\0`, `\n`, `\r`, `\`, `'`, dan `"`, sedangkan kolom biner (`BINARY`, `VARBINARY`, `BLOB`) dikonversi ke format heksadesimal `0x...` agar bebas dari korupsi *encoding*.
4. View Dependency & Post-Data Hook	Mengeksekusi pembuatan objek non-tabel setelah seluruh data tabel selesai disalin dengan urutan ketat: (1) *Placeholder Views*, (2) *Final Views*, (3) *Procedures & Functions*, (4) *Triggers*, dan (5) *Events*.	**Mengapa Trigger dibuat paling akhir?** Jika *Trigger* dibuat sebelum data disalin, setiap perintah *Bulk INSERT* ke tabel target akan memicu *trigger* `BEFORE/AFTER INSERT` dan merusak integritas data. Untuk *View* yang saling bergantung, mesin membuat *dummy table* sementara lalu menggantinya dengan `CREATE OR REPLACE VIEW`.

Modul 7: Streamed SQL Dump, Script Executor & Import/Export CSV/JSON

(`DataMigrationTools`)

Menyediakan utilitas pencadangan dan pemulihan berkas lokal yang dibangun di atas mesin *streaming* yang sama dengan Modul 6 sehingga tidak pernah mengalami *Out-Of-Memory* pada berkas berukuran besar.

- **Backup / Dump Database as SQL (`.sql`)**: Menghasilkan berkas *dump* SQL standar yang memuat *header* informasi server, pengaturan `SET FOREIGN_KEY_CHECKS=0;`, DDL objek, serta pernyataan *extended bulk INSERT* berukuran terukur (default 4 MB per pernyataan `INSERT`). Mendukung kompresi langsung ke `.sql.gz` (*gzip stream*) di sisi Rust menggunakan crate `flate2`.
- **Streamed SQL Script Executor**: Mengeksekusi berkas `.sql` atau `.sql.gz` berukuran puluhan gigabita tanpa membuka berkas tersebut di editor teks. Mesin Rust membaca berkas dalam potongan 64 KB (`BufReader`), memisahkan pernyataan SQL dengan *state machine* yang mengenali `DELIMITER`, mengeksekusinya secara berurutan ke server target, dan melaporkan persentase *byte offset* berkas yang telah diproses ke bilah progres C#.
- **Import & Export CSV / JSON Ringan**: Menyediakan jendela satu langkah untuk mengekspor tabel atau hasil kueri ke `.csv` / `.json` (*JSON Array* maupun *JSON Lines / NDJSON*), serta mengimpor berkas `.csv` / `.json` ke tabel baru atau tabel yang sudah ada dengan pratinjau 50 baris pertama, deteksi tipe kolom otomatis, dan mode impor (*Append*, *Replace/Truncate + Insert*, atau *Insert Ignore / Upsert*).

Modul 8: Server Process Monitor & User Privilege Manager (`AdminTools`)

Melengkapi kebutuhan administrasi harian DBA dengan perkakas pemantauan sesi server dan pengelolaan hak akses pengguna yang ringkas:

Sub-Modul Administrasi	Tampilan & Kontrol Antarmuka (C#)	Perintah SQL Internal & Fitur Keamanan (Rust)
Server Process Monitor	Tabel proses *real-time* dengan kolom `Id`, `User`, `Host`, `db`, `Command`, `Time (s)`, `State`, dan `Info (Full SQL)`; dilengkapi pengatur *Auto-Refresh* (1s / 3s / 5s / Manual), kotak filter teks, dan tombol **Kill Query** / **Kill Connection**.	Menjalankan `SHOW FULL PROCESSLIST;` (atau membaca `information_schema.PROCESSLIST`). Tombol penghentian mengeksekusi `KILL QUERY <id>;` (menghentikan kueri tanpa memutus koneksi aplikasi klien) atau `KILL CONNECTION <id>;`.
User & Privilege Manager	Daftar pengguna MySQL (`'user'@'host'`) di sisi kiri; panel kanan memuat tab *General* (Sandi, Plugin Autentikasi `caching_sha2_password` / `mysql_native_password`, Batas Koneksi/Jam), tab *Global Privileges*, dan tab *Schema/Table Privileges*.	Membaca `mysql.user` dan `SHOW GRANTS FOR 'user'@'host';`, lalu menghasilkan pernyataan `CREATE USER`, `ALTER USER`, `GRANT ... ON db.tbl TO 'user'@'host'`, `REVOKE`, serta tombol cepat `FLUSH PRIVILEGES;`.